



Figure 1: Sponza Scene with a direct light and a blue point light.

Vulkan-Based Real-Time Rendering Engine

Pedro Freire

March 14, 2026

Contents

1	Summary	3
2	Goals and Design Philosophy	4
2.1	Explicit Control	4
2.2	Modularity	4
2.3	Scalability and Extensibility	4
2.4	Learning Objectives	4
3	High-Level-Architecture	5
4	Module Breakdown	7
4.1	Renderer	7
4.2	Engine	8
4.3	Shaders	9
5	Rendering Techniques	11
5.1	Rendering Pipeline	11
5.2	Physically Based Rendering	11
6	Image-Based Lighting	12
7	glTF Asset Import	13
8	Performance	15
9	Tools and Debugging	16
10	Limitations and Future Improvements	17
11	Conclusion	18

1 Summary

This project is a personal real-time graphics engine written in C++ using the Vulkan graphics API. The goal of the project is to explore modern low-level graphics programming while developing a modular engine architecture capable of supporting advanced rendering techniques. While the long-term objective is to expand the project into a more complete game engine, the current focus is primarily on building a robust and extensible rendering system.

The graphics engine makes extensive use of RAII abstractions to manage Vulkan resources safely through object lifetimes. Factory and manager systems are used to simplify the creation and configuration of complex Vulkan objects such as pipelines, descriptor sets, and GPU resources, resulting in easier GPU resource management.

The graphics engine implements physically based rendering using the metallic-roughness workflow. Implemented features include normal mapping, emissive and ambient occlusion textures, and image-based lighting (IBL). Environment maps are generated from equirectangular HDR images, and the required lighting resources irradiance maps, prefiltered environment maps, and a BRDF lookup table—are computed on the GPU simultaneously.

Scene data is loaded using the glTF 2.0 format and represented internally through a scene graph that remains independent of the rendering backend. Each frame, the scene generates draw submissions that the renderer resolves into the corresponding GPU resources and Vulkan draw calls..

While the long-term goal is to expand the project into a more complete engine, the current focus is on developing a robust and extensible rendering system.

Key Features:

- C++ engine architecture
- RAII resource management for Vulkan objects
- Physically Based Rendering (metallic-roughness workflow)
- Image-Based Lighting (IBL)
- GPU generation of irradiance maps, prefiltered environment maps, and BRDF LUT
- Normal mapping, emissive, and ambient occlusion textures
- glTF scene loading
- API-agnostic scene graph

2 Goals and Design Philosophy

The primary objective of this project is to explore modern graphics programming and engine architecture through the implementation of a real-time renderer using the Vulkan graphics API. The project serves both as a learning platform and as a foundation for experimenting with advanced rendering techniques and engine architecture.

2.1 Explicit Control

Vulkan was chosen as the primary graphics API due to its explicit design and fine-grained control over GPU resources. Working with a low-level API enables deeper understanding of GPU synchronisation, command submission, and memory management, which are essential for performant graphics programming.

2.2 Modularity

Scene representation, asset management, and other engine systems such as the scene graph and resource management remain independent of the graphics API, while the renderer is responsible for translating this data into Vulkan-specific commands. This separation helps maintain clear system boundaries and improves maintainability.

2.3 Scalability and Extensibility

The architecture is intended to support the gradual addition of new rendering techniques without requiring major structural changes. Systems such as the resource manager, pipeline abstractions, and scene representation are designed to make it easier to integrate additional features such as shadow mapping, post-processing, or alternative rendering pipelines.

2.4 Learning Objectives

A central motivation behind the project is to gain practical experience with the core concepts behind modern game engines. Key areas of exploration include:

- GPU pipeline configuration and rendering workflows
- Resource and memory management in explicit graphics APIs
- GPU–CPU synchronization and frame scheduling
- Structuring a renderer within a larger engine architecture

3 High-Level-Architecture

In its current state, the engine primarily functions as a rendering system. However, several supporting systems and structural components already exist outside the renderer.

The project is organized into three main directories: `renderer`, `shaders`, and `engine`.

The overall folder structure is as follows:

```
Engine/
├── Core/
│   ├── defines.h
├── Loaders/
│   ├── gltfloader
├── Resources/
│   ├── ResourceManager
├── Scene/
│   ├── camera
│   └── Scene
├── UI/
│   └── imguimanager
├── Renderer/
│   ├── Includes/
│   ├── Platform/
│   │   └── window
│   ├── Renderer/
│   │   ├── CommandBufferRecorder
│   │   ├── DescriptorManager
│   │   ├── PipelineFactory
│   │   └── VulkanRenderer
│   └── Vulkan/
│       └── RAII wrappers for Vulkan objects
├── Shaders/
│   ├── Compiled/
│   │   └── Mirrors shader folder structure
│   ├── environment/
│   │   ├── envMap
│   │   ├── equiToCube
│   │   └── ibl
│   ├── include/
│   └── PBR/
│       └── forwardPBR
```

The `renderer` directory contains all rendering-related functionality. All graphics API interactions are encapsulated within this module to ensure that GPU control remains isolated from the rest of the engine.

In the current implementation, most API calls are already restricted to the `renderer`. The only remaining exception is part of the resource management

system, which still performs limited API-related operations. This will be migrated into the renderer module as the resource pipeline is further refined.

The shaders directory contains the GLSL shader source files used by the engine. These are organized into subdirectories according to their purpose. In addition, a compiled directory mirrors the same folder structure and stores the compiled SPIR-V (.spv) shader binaries.

The engine directory currently contains the core non-rendering systems. These include the scene graph implementation, the loader for glTF 2.0 scenes, the scene resource manager, and the UI control system. Although this module is still relatively minimal, it will eventually host additional high-level engine functionality.

The defines.h file currently acts as a central aggregation header containing commonly used definitions and includes shared across multiple engine components. While this simplified development during the early stages of the project, it is currently technical debt and will be refactored into smaller, module-specific headers as the architecture continues to mature.

The frame update process begins within the engine systems. During this stage, the scene graph is traversed to generate a list of draw calls representing all renderable objects currently present in the scene. At this stage, I have not implemented view culling, so draw calls are generated for every object in the scene.

The UI system is then updated, and the resource manager performs any required per-frame operations, such as updating assets.

Once these steps are complete, the generated draw call list is submitted to the renderer, which is responsible for translating these commands into the appropriate GPU operations required to produce the final frame.

4 Module Breakdown

4.1 Renderer

The renderer module encapsulates all graphics API interactions and is responsible for translating high-level draw calls generated by the engine into GPU commands. The module is implemented using Vulkan and provides abstractions for pipeline and render pass creation, descriptor management, and command buffer recording.

Vulkan is a very verbose API that requires explicit GPU memory management. To simplify usage, a collection of wrappers and auxiliary classes was developed around the core Vulkan objects. These abstractions facilitate development by reducing boilerplate and clarifying responsibilities.

Vulkan RAI Wrappers The first layer of abstraction consists of wrapper classes around Vulkan base types. These wrappers follow the RAI (Resource Acquisition Is Initialization) design pattern. This means that when a wrapper is created, the underlying Vulkan object is also created and its lifetime is tied to that of the wrapper. This approach simplifies resource management and ensures proper cleanup of GPU memory.

Vulkan object creation typically involves providing both a handle and a verbose configuration structure, which can vary significantly between object types. For objects with complex setup requirements, the wrappers either accept a fully pre-configured structure or provide simplified custom structures.

When the base structure is used, an additional helper class assists in its setup. Examples include descriptor sets, command buffers and their pools, pipelines, and render passes. For simpler cases, such as images, custom structures with default values are used to significantly reduce setup complexity.

Resource Creation As mentioned, some wrappers rely on additional helper functions for resource creation. These functions are organised in stateless classes, each containing methods to configure and initialise different Vulkan objects. They take the necessary data for setup—including access to the renderer itself—and the object to be created.

In these cases, the RAI principle is temporarily relaxed, since object creation occurs when the initialisation function is called rather than at wrapper construction. This is a known limitation and is planned to be addressed in future refactoring.

Despite this exception, these helper interfaces greatly simplify object creation by hiding Vulkan’s verbose boilerplate code, making the renderer easier to understand, maintain, and extend in the future.

Frame Execution The frame execution system is responsible for processing the list of draw calls generated by the engine and submitting them to the GPU. Each frame begins with traversing the scene graph to produce draw calls for all objects in the scene. Currently, view frustum culling has not been implemented, so draw calls are generated for every renderable object.

Once the draw call list is ready, the UI system is updated, and the resource

manager performs any required per-frame operations, such as loading or updating assets. These steps ensure that all resources are prepared before rendering begins.

The draw calls are then submitted to the renderer, which records the necessary commands into command buffers and submits them to the GPU. This process coordinates pipeline state, descriptor sets, and synchronization primitives to guarantee that rendering occurs efficiently and correctly for each frame.

Although draw calls are not currently submitted using GPU instancing, the draw call data generated by the engine already contains a list of instances per submesh and material. This design allows instancing to be implemented without a large refactor.

4.2 Engine

The engine module is still very light, currently it only contains the scene and its required functions, other functions like physics and animations and an ECS system to connect all those to the scene has not been implemented yet.

Scene The scene is composed of multiple nodes. Each node may reference zero or more assets, such as a mesh, light, or camera. Nodes are organised into one or more hierarchical structures called scene graphs, which represent the spatial relationships between objects. These scene graphs are typically implemented as tree structures, where transformations applied to parent nodes affect their children.

During frame processing, the scene graph is traversed to collect renderable objects and generate the corresponding draw calls. This traversal provides the renderer with the information required to render each object, including mesh data, material references, lights, cameras and transformation matrices. Each node stores a local transform, which is combined with its parent transform during traversal to compute the final world-space transformation.

Resource Manager The resource manager is created after the scene graph and is responsible for the creation and lifetime management of GPU resources that represent the scene. These resources include objects such as meshes, textures, materials, and other data required by the renderer.

In addition to managing standard assets, the resource manager also handles the generation of auxiliary rendering resources and performs their precomputation when applicable. An example of this is the set of textures required for image-based lighting (IBL), such as irradiance maps and prefiltered environment maps. In the future, this system will also manage additional precomputed data such as shadow maps and other rendering-related resources.

Currently, the resource manager performs some API-specific operations during resource creation. In future revisions, these creation functions should be abstracted behind an `IRenderer` interface, allowing resource ownership to remain independent of the underlying graphics API. This approach would improve modularity, simplify future refactoring, and allow the engine to support multiple rendering backends.

Asset Loading Asset loading is performed using the `glTFLoader` library, which imports scene data from the glTF 2.0 format. This format provides a standardized representation for 3D assets, including meshes, materials, textures, lights, and scene hierarchies.

During the loading process, the glTF file is parsed and its contents are converted into the engine’s internal asset structures. These assets are then inserted into the scene graph. At this stage, the assets exist as CPU-side data structures and are later registered with the resource manager, which creates the corresponding GPU resources required for rendering.

The loader supports several glTF extensions that extend the material and lighting capabilities of the format. These include punctual light sources, compressed textures using the Basis Universal format, and additional physically based rendering (PBR) parameters such as index of refraction, specular properties, transmission, and emissive strength.

To improve loading performance, texture processing is partially parallelised using OpenMP. This allows multiple textures to be loaded and prepared simultaneously, significantly reducing the overall asset loading time when scenes contain a large number of textures.

UI The engine includes a lightweight user interface system implemented using the Dear ImGui library. Currently, the UI displays only basic performance metrics, namely frames per second (FPS), frame time, and a frame time graph.

In future iterations, the interface will be expanded to include controls for selecting scenes and environment textures at the beginning of the application rather than relying on hardcoded configuration. Additional diagnostic tools, such as GPU performance monitoring and resource statistics, may also be integrated to assist with profiling and debugging the rendering pipeline.

4.3 Shaders

The shaders module contains the GLSL shader programs used by the renderer as well as shared shader utilities. The directory is organised according to shader functionality and rendering stages.

The `environment` directory contains shaders used for environment map processing and image-based lighting preparation. These include shaders for rendering the environment map, converting equirectangular textures to cubemaps, and generating the textures required for image-based lighting.

The `PBR` directory contains the main forward rendering shader implementing the physically based shading model used by the renderer.

Shared shader code is located in the `includes` directory. This directory contains reusable GLSL utilities that are included by multiple shaders. In particular, the `brdf` module contains implementations of GGX normal distribution function, the Schlick Fresnel approximation, and the Geometry Schlick visibility term. Additional sampling utilities are also provided, including functions such as `RadicalInverse_VdC`, `Hammersley`, and `ImportanceSampleGGX`, which are used during environment map precomputation and importance sampling operations.

Finally, the `Compiled` directory mirrors the structure of the shader source folders and contains the compiled SPIR-V shader binaries used by the Vulkan renderer at runtime.

5 Rendering Techniques

5.1 Rendering Pipeline

The current rendering pipeline is implemented as a forward rendering pipeline. During renderer initialization, the required Vulkan graphics pipeline and render pass objects are created. Dynamic rendering is not currently used, therefore the render pass configuration is defined explicitly at setup time.

The render pass consists of two subpasses. The first subpass renders the environment map, which provides the background. The second subpass performs the main forward rendering pass, where scene geometry is drawn using physically based shading.

During each frame, an appropriate swapchain image is acquired and the corresponding command buffer is recorded. The command buffer contains the rendering commands required to execute both subpasses. Since the renderer does not currently support GPU instancing, a separate draw call is recorded for each instance of an object, even when multiple instances share the same mesh and material.

The swapchain is configured using the FIFO presentation mode. In this mode, rendered images are placed in a presentation queue and displayed in the order they were submitted. This behaviour is similar to traditional vertical synchronisation (VSync), ensuring that images are presented in a consistent order and preventing screen tearing.

If supported by the hardware, the renderer enables the `VK_PRESENT_MODE_MAILBOX_KHR` presentation mode. In this mode, newly rendered frames replace older queued frames that have not yet been presented. This behaviour reduces presentation latency while still preventing screen tearing.

Synchronization To synchronize rendering with the swapchain, a structure containing three vectors of synchronization primitives is used. Each vector has the same size as the number of swapchain images, resulting in three synchronization objects per swapchain image.

The `imageAvailableSemaphores` vector contains semaphores that signal when a swapchain image has finished presentation and is available for rendering. The `renderFinishedSemaphores` vector contains semaphores that signal when rendering to the image has completed, ensuring that the image is not presented before all rendering commands have finished execution.

The `inFlightFences` vector is used to synchronize CPU and GPU execution. Before recording commands for a new frame, the CPU waits on the corresponding fence to ensure that the GPU has finished executing the commands from the previous frame that used the same resources.

The engine also does offline tasks in parallel, these include the loading of images and the generation of IBL resources.

5.2 Physically Based Rendering

The renderer implements a physically based rendering (PBR) model based on the metallic-roughness workflow commonly used in modern real-time graphics. In

this approach, materials are described using a small set of parameters such as base color, metallic value, roughness, and normal information. These parameters allow a wide range of realistic materials to be represented while maintaining physically plausible interactions with light.

Lighting is evaluated using a microfacet Bidirectional Reflectance Distribution Function (BRDF). The implementation follows the Cook–Torrance model. The specular BRDF is expressed as:

$$f_{\text{specular}} = \frac{D \cdot G \cdot F}{4(N \cdot L)(N \cdot V)}$$

where N is the surface normal, L is the light direction, V is the view direction, and H is the half-vector between L and V . The terms D , G , and F correspond to the normal distribution function, the geometry term, and the Fresnel term respectively.

The normal distribution function models the statistical distribution of microfacets on the surface. In this implementation the GGX distribution is used:

$$D_{\text{GGX}}(N, H, \alpha) = \frac{\alpha^2}{\pi ((N \cdot H)^2(\alpha^2 - 1) + 1)^2}$$

where α represents the surface roughness parameter.

The geometry term accounts for shadowing and masking effects between microfacets. This is approximated using the Schlick–GGX formulation:

$$G_{\text{SchlickGGX}}(N, V, k) = \frac{N \cdot V}{(N \cdot V)(1 - k) + k}$$

where k is derived from the material roughness.

The Fresnel term models how reflectance increases as the viewing angle approaches grazing angles. It is approximated using the Schlick Fresnel equation:

$$F_{\text{Schlick}} = F_0 + (1 - F_0)(1 - V \cdot H)^5$$

where F_0 represents the base reflectance of the material.

These functions are implemented in the shader utilities contained in the `brdf.gls1` module. The fragment shader combines the distribution, geometry, and Fresnel terms to compute the final lighting contribution for each fragment.

The implementation of the PBR model was conceptually guided by the PBR chapter of the LearnOpenGL resource. While this material served as a reference for the underlying shading principles, the entire implementation was developed specifically for this engine and adapted for Vulkan, integrating the BRDF evaluation with the renderer’s shader pipeline and resource management system.

6 Image-Based Lighting

In addition to direct lighting, the renderer uses image-based lighting (IBL) to approximate indirect lighting from the surrounding environment. Instead of relying

solely on analytic light sources, IBL uses an environment map representing incoming radiance from all directions to illuminate objects in the scene.

The environment lighting contribution is separated into diffuse and specular components. The diffuse component is computed using an irradiance map, which represents the incoming diffuse light integrated over the hemisphere around the surface normal. This map is generated by convolving the environment map with a diffuse Lambertian response.

The specular component is more complex because it depends on the surface roughness. To approximate this efficiently in real time, a prefiltered environment map is generated where different mip levels correspond to different surface roughness values. During rendering, the appropriate mip level is sampled based on the material roughness.

To complete the specular IBL calculation, a two-dimensional BRDF lookup texture (BRDF LUT) is used. This texture stores the result of integrating the microfacet BRDF over the hemisphere for varying view angles and roughness values. Using this lookup table avoids performing expensive numerical integration during rendering.

The precomputation of these textures is performed using dedicated shaders contained in the `environment` module. The environment map is first converted from an equirectangular HDR texture into a cubemap representation. From this cubemap, an irradiance map and a prefiltered environment map are generated.

Importance sampling is used during the prefiltering process to efficiently sample the hemisphere according to the GGX distribution. The implementation relies on helper functions defined in `samplingHelpers.glsl`, including `RadicalInverse_VdC`, `Hammersley`, and `ImportanceSampleGGX`, which generate quasi-random sample sequences for importance sampling.

The overall IBL pipeline therefore consists of the following steps:

1. Conversion of an equirectangular HDR environment texture to a cubemap
2. Generation of a diffuse irradiance map
3. Generation of a prefiltered specular environment map
4. Precomputation of the BRDF lookup texture

These precomputed textures are then used during the forward rendering pass to provide realistic reflections and ambient lighting for physically based materials.

The implementation of this image-based lighting pipeline was inspired by the IBL section of LearnOpenGL. The techniques were adapted to fit the Vulkan rendering architecture and integrated with the engine's shader system and resource manager.

7 glTF Asset Import

The engine uses the glTF 2.0 format for scene and asset import. glTF is a widely used open standard designed for efficient transmission of 3D scenes and

supports physically based materials, hierarchical scene structures, and compact asset storage, making it well suited for real-time rendering applications.

Asset loading is implemented using the `fastGltf` library. During the loading process, the glTF file is parsed and its contents are converted into the engine's internal asset structures in the scene object. The loader reconstructs the scene hierarchy by creating scene nodes that preserve the transformations defined in the glTF file while extracting meshes, materials, textures and lights.

Once loaded, these assets are registered with the engine's resource manager, which is responsible for creating the corresponding GPU resources such as buffers, textures, and material descriptors.

The loader currently supports several glTF extensions used in physically based rendering pipelines, including `KHR_lights_punctual`, `KHR_texture_basisu`, `KHR_materials_ior`, `KHR_materials_specular`, `KHR_materials_transmission`, and `KHR_materials_emissive_strength`. Texture loading is parallelized using OpenMP to reduce loading times for scenes containing multiple textures.

8 Performance

At the current stage of development, only limited performance metrics are collected. The engine measures frame time and frames per second (FPS), which are displayed through the integrated ImGui interface.

These metrics provide a basic indication of rendering performance but do not yet offer detailed insight into GPU or CPU bottlenecks.

The renderer has been tested using moderately complex scenes such as the Sponza scene, where the engine maintains interactive frame rates on modern hardware.

On a laptop equipped with an NVIDIA RTX 4060 GPU, the engine maintains frame rates between approximately 130 and 140 frames per second when rendering the Sponza scene at 1920×1080 resolution.

Since features such as view frustum culling, GPU instancing, and resource streaming are not yet implemented, the current performance should be considered preliminary.

9 Tools and Debugging

Development relied on several tools to validate and debug the renderer. Vulkan validation layers were used to catch API misuse and synchronization issues, while RenderDoc allowed inspection of frame captures, draw calls, and shader inputs. Visual Studio profiling tools were also used to measure CPU performance and identify bottlenecks.

An in-engine Dear ImGui overlay displays basic runtime metrics such as FPS and frame time for quick performance monitoring.

10 Limitations and Future Improvements

The engine is still an early-stage prototype and lacks several important features. The renderer currently does not implement view frustum culling, which means draw calls are generated for all objects in the scene regardless of visibility. GPU instancing is also not yet supported, so each instance is submitted as a separate draw call.

Performance instrumentation is currently limited to basic frame timing metrics, and detailed GPU profiling has not yet been implemented.

Additionally, the renderer does not yet support shadow rendering and lacks a render graph system to manage render passes and resource dependencies. These features represent important areas for future development.

11 Conclusion

Listings

List of Tables

List of Figures

1	Sponza Scene with a direct light and a blue point light.	1
---	--	---